

Semana 5 - Implementado regra de detecção de Brute Force e sistema de alertas

O que foi feito

Nesta semana foi implementado o sistema de detecção de ataques de força bruta SSH e o módulo de alertas de segurança. O foco foi criar o pipeline completo: receber um log de autenticação, detectar padrão de brute force e gerar automaticamente um alerta persistido no banco — tudo sem interromper o fluxo normal de recebimento de logs.

Arquivos criados ou alterados

Arquivo	Ação	Descrição
app/models/alert.py	Criado	Modelo Alert — tabela alerts
app/models/__init__.py	Alterado	Registra AlertModel em registrar_modelos()
app/utils/detection.py	Criado	Função check_brute_force()
app/utils/__init__.py	Alterado	Exporta check_brute_force
app/routes/alerts.py	Criado	Endpoints GET e PATCH /api/alerts
app/routes/__init__.py	Alterado	Exporta register_alerts_routes
app/routes/logs.py	Alterado	Chama check_brute_force() após log failed
app/app.py	Alterado	Registra AlertModel, garantir_schema_alerts()

app/models/alert.py

Modelo SQLAlchemy para a tabela alerts, seguindo o padrão de wrapper com get_model(db) adotado no projeto desde a Semana 2.

Campos da tabela

Campo	Descrição
id	BIGSERIAL — chave primária gerada pelo banco
host_id	FK para host.id — host que sofreu o ataque
alert_type	Tipo do alerta: "brute_force" ou "port_scan"
source_ip	IP de origem do ataque (string, 45 chars)
timestamp	Momento em que o alerta foi gerado (default: utcnow)
severity	Severidade: "low", "medium", "high", "critical" (default: "medium")
metodos	Método de ataque observado, ex: "password" (herdado do schema da Beatriz)
resolved	Boolean — False = alerta ativo, True = resolvido (default: False)
resolved_at	DateTime — momento da resolução, nullable

Observação: os campos resolved e resolved_at não estavam no init.sql original. A função garantir_schema_alerts() em app.py adiciona essas colunas automaticamente via ALTER TABLE IF NOT EXISTS, sem exigir recriação do container.

app/utils/detection.py

Módulo de detecção de ameaças. Contém a função check_brute_force().

check_brute_force(db, LogEntryModel, AlertModel, host_id, ip_origem)

Fluxo de execução:

- Chama count_failed_logins() para contar falhas SSH do IP nos últimos 60 segundos (janela_horas = 1/60).
- Se o total de falhas for menor que 5 (LIMIAR_BRUTE_FORCE), retorna False imediatamente.
- Se atingir ou superar o limiar, consulta se já existe um alerta ativo (resolved = False) para a combinação host_id + source_ip + alert_type = "brute_force".

- Se já existir alerta ativo: não cria duplicata, retorna False.
- Se não existir: cria novo AlertModel com severity = "high" e metodos = "password", salva no banco e retorna True.
- Em caso de qualquer exceção interna: rollback silencioso, retorna False — o fluxo de recebimento de logs não é interrompido.

Constantes configuráveis

Constante	Valor / Descrição
LIMIAR_BRUTE_FORCE	5 — mínimo de falhas para disparar alerta
JANELA_HORAS	1/60 — janela de 60 segundos

app/routes/alerts.py

Blueprint de alertas registrado via register_alerts_routes(). Expõe dois endpoints para consulta e resolução de alertas.

GET /api/alerts

Retorna alertas de segurança com filtros opcionais.

Parâmetros (query string):

Parâmetro	Descrição
status	"active" (padrão), "resolved" ou "all"
host_id	Filtra por host específico (opcional)
limit	Máximo de registros retornados (padrão: 20, máximo: 100)
offset	Deslocamento para paginação (padrão: 0)

Exemplo de resposta (status=active):

```
{
  "alerts": [
    {
      "id": 1,
      "host_id": 2,
      "alert_type": "brute_force",
      "source_ip": "192.168.1.100",
      "timestamp": "2026-05-21T03:14:00",
      "severity": "high",
      "metodos": "password",
      "resolved": false,
      "resolved_at": null
    }
  ],
  "total": 1,
  "status": "active",
  "limit": 20,
  "offset": 0
}
```

Código HTTP: 200 OK**PATCH /api/alerts/<id>/resolve**

Marca um alerta como resolvido. Define resolved = True e resolved_at = datetime.utcnow().

Comportamento:

- 404 Not Found — se o alerta não existir.

- 400 Bad Request — se o alerta já estiver resolvido (evita dupla resolução).
- 200 OK — retorna o alerta atualizado com resolved_at preenchido.

Exemplo de resposta (200 OK):

```
{  
  "message": "Alerta 1 resolvido com sucesso",  
  "alert": {  
    "id": 1,  
    "resolved": true,  
    "resolved_at": "2026-05-21T10:30:00",  
    ...  
  }  
}
```

Código HTTP: 200 OK

app/routes/logs.py

O endpoint POST /api/logs foi atualizado para acionar a detecção de brute force automaticamente após salvar um log de autenticação SSH com falha.

Mudanças aplicadas:

- register_log_routes() passou a receber AlertModel como parâmetro adicional.
- Após o db.session.commit() do log, se log_type == "auth" e parsed_data['status'] == "failed", chama check_brute_force(db, LogEntryModel, AlertModel, host_id, ip_origem).
- check_brute_force() é chamado APÓS o commit para garantir que a falha recém-salva já seja contabilizada na janela de 60 segundos.
- Falhas internas da detecção são silenciosas — nunca afetam o retorno 201 do endpoint de logs.
- A resposta JSON do POST /api/logs ganhou o campo alerta_criado (bool) para informar ao chamador se um novo alerta foi disparado.

app/app.py

Mudanças aplicadas:

- registrar_modelos(db) agora retorna 6 valores — AlertModel foi adicionado ao desempacotamento.
- garantir_schema_alerts() adicionada: executa ALTER TABLE alerts ADD COLUMN IF NOT EXISTS resolved e resolved_at, garantindo compatibilidade com bancos criados antes desta semana.
- register_alerts_routes(app, db, HostModel, AlertModel) registrado.
- register_log_routes() atualizado para receber AlertModel como argumento.
- Versão da API atualizada para 3.0.0 em GET /api/status.
- Print de inicialização atualizado com os dois novos endpoints.

Endpoints implementados nesta semana

Método + Rota	Semana	Descrição
GET /api/alerts	5	Lista alertas com filtro de status e host
PATCH /api/alerts/<id>/resolve	5	Marca alerta como resolvido

Observações técnicas

- A tabela alerts foi criada pelo init.sql da Beatriz. O backend não usa db.create_all() — as colunas resolved e resolved_at são adicionadas via garantir_schema_alerts() com ALTER TABLE IF NOT EXISTS.
- check_brute_force() é chamado APÓS o commit do log para que a falha recém-gravada já seja contabilizada pela query count_failed_logins(), que usa operadores JSONB nativos do PostgreSQL aproveitando os índices idx_logs_auth_ip e idx_logs_auth_failed.

- O limiar de 5 falhas em 60 segundos e a severidade "high" são constantes em detection.py — fáceis de ajustar sem alterar a lógica de negócio.
- A injeção de modelos por parâmetro (LogEntryModel, AlertModel passados para as funções) foi mantida em todos os módulos novos para evitar import circular com app.py, seguindo o padrão já estabelecido nas semanas anteriores.
- O campo alerta_criado na resposta de POST /api/logs é útil para o agente ou para testes — permite saber, na mesma requisição, se um alerta foi disparado.

Pendências para a próxima semana

- Implementar detecção de port scan (alert_type = "port_scan") utilizando a tabela active_connections já criada no banco.
- Avaliar limiar e janela de tempo para brute force com base em dados reais coletados pelo agente.
- Discutir com a equipe de frontend o contrato do endpoint GET /api/alerts para exibição no dashboard.
- Verificar necessidade de autenticação nos endpoints de alertas (prevista para a Semana 7).